
Manual Técnico 11

NFS, iSCSI e Object Storage com MinIO

Infraestruturas de Computação na Nuvem | Aula 11

Como usar este manual

Vai configurar três formas distintas de armazenamento em rede e, sobretudo, perceber *porque* diferem. O momento mais importante está na secção 4: montar a mesma partilha NFS em dois sítios funciona; fazer o mesmo com iSCSI corromperia os dados. Perceber essa diferença é perceber a distinção entre ficheiro e bloco.

Atenção

Precisa de **duas VMs** nesta aula: uma faz de *servidor*, outra de *cliente*. Cada bloco de comandos indica onde correr:

- **[SRV]** – na VM servidor
- **[CLI]** – na VM cliente

Antes de começar

- ☐ Duas VMs Linux ligadas, na mesma rede, com IPs registados
- ☐ Acesso `sudo` em ambas
- ☐ Docker na VM servidor (para o MinIO)
- ☐ Opcional: cluster Kubernetes da Aula 05, para a Parte E

Anote antes de começar:

```
IP do servidor : ____·____·____·____  
IP do cliente  : ____·____·____·____
```

[SRV]

```
$ sudo apt update  
$ sudo apt install -y nfs-kernel-server targetcli-fb
```

[CLI]

```
$ sudo apt update  
$ sudo apt install -y nfs-common open-iscsi
```

1 Parte A – Storage de rede no Proxmox

Na interface web: **Datacenter** → **Storage** → **Add**. Consulte a lista de tipos disponíveis **sem adicionar nada** e identifique os de rede: *NFS*, *iSCSI*, *CIFS/SMB*, *Ceph RBD*.

```
$ pvesm status
```

Pergunta para registo: se o disco de uma VM estiver num storage *local*, o que teria de acontecer para a migrar para outro nó? E se estivesse num storage *partilhado*? Retome o conceito de migração ao vivo da Aula 03 – é a razão pela qual o storage partilhado existe.

2 Parte B – NFS: partilha ao nível de ficheiro

2.1 Preparar a pasta

[SRV]

```
$ sudo mkdir -p /srv/nfs/partilha
$ sudo chown nobody:nogroup /srv/nfs/partilha
$ sudo chmod 777 /srv/nfs/partilha
```

2.2 Exportar

[SRV]

```
$ sudo nano /etc/exports
```

Acrescente (ajustando a sub-rede à sua):

```
/srv/nfs/partilha 192.168.1.0/24(rw,sync,no_subtree_check)
```

As opções: **rw** permite escrita; **sync** confirma as escritas em disco antes de responder (mais seguro, mais lento); **no_subtree_check** evita verificações desnecessárias que causam problemas quando ficheiros são renomeados.

[SRV]

```
$ sudo exportfs -ra
$ sudo systemctl restart nfs-kernel-server
$ sudo exportfs -v
```

Saída esperada

```
/srv/nfs/partilha 192.168.1.0/24(sync,wdelay,hide,no_subtree_check,sec=sys,rw,...)
```

2.3 Montar no cliente

[CLI]

```
$ sudo mkdir -p /mnt/nfs
$ sudo mount <IP-do-servidor>:/srv/nfs/partilha /mnt/nfs
$ df -h /mnt/nfs
```

Saída esperada

```
Filesystem Size Used Avail Use% Mounted on
192.168.1.50:/srv/nfs/partilha 20G 4.2G 15G 23% /mnt/nfs
```

Confirme antes de avançar

O `df` mostra a partilha montada, com o endereço do servidor no lugar do dispositivo. Repare que **não formatou nada** – o sistema de ficheiros já existia, do lado do servidor.

2.4 Testar a escrita

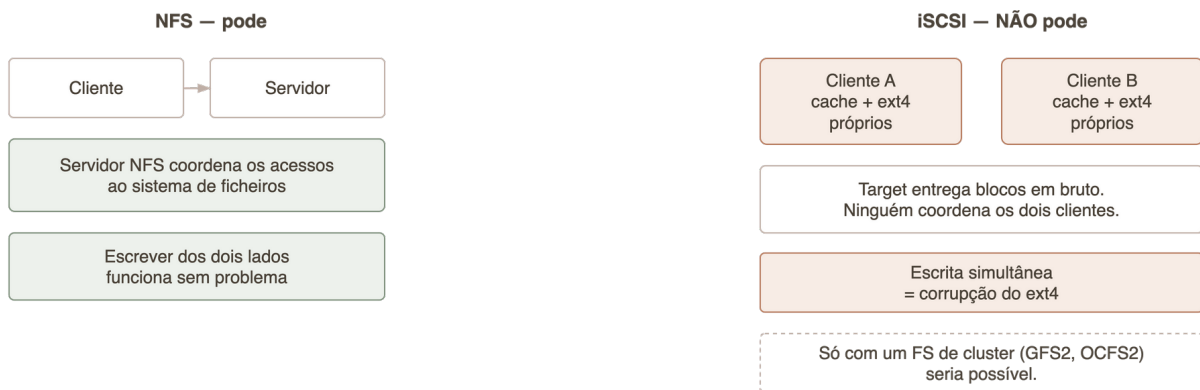
[CLI]

```
$ echo "ola via nfs" | sudo tee /mnt/nfs/teste.txt
```

[SRV]

```
$ cat /srv/nfs/partilha/teste.txt
```

2.5 Acesso concorrente – o teste importante



A diferença que distingue armazenamento de ficheiro de armazenamento de bloco.

Monte a mesma partilha **também no servidor**, num ponto diferente:

[SRV]

```
$ sudo mkdir -p /mnt/nfs-local
$ sudo mount <IP-do-servidor>:/srv/nfs/partilha /mnt/nfs-local
```

Agora escreva dos dois lados alternadamente:

[CLI]

```
$ echo "linha escrita pelo cliente" | sudo tee /mnt/nfs/concorrente.txt
```

[SRV]

```
$ cat /mnt/nfs-local/concorrente.txt
$ echo "linha acrescentada pelo servidor" | sudo tee -a /mnt/nfs-local/concorrente.txt
```

[CLI]

```
$ cat /mnt/nfs/concorrente.txt
```

Saída esperada

```
linha escrita pelo cliente
linha acrescentada pelo servidor
```

Confirme antes de avançar

Funcionou. Ambos os lados veem as alterações um do outro, sem corrupção. Porquê? Porque é o *servidor NFS* que gere o sistema de ficheiros e coordena os acessos. Guarde esta observação – vamos contrastá-la já a seguir.

3 Parte C – iSCSI: armazenamento ao nível de bloco

3.1 Criar o disco virtual

[SRV]

```
$ sudo mkdir -p /srv/iscsi
$ sudo truncate -s 500M /srv/iscsi/disco0.img
$ ls -lh /srv/iscsi/
```

3.2 Configurar o target

[SRV]

```
$ sudo targetcli
```

Dentro da consola do `targetcli`, escreva cada linha:

```
/> backstores/fileio create disco0 /srv/iscsi/disco0.img
/> iscsi/ create iqn.2026-11.icn.estga:servidor
/> iscsi/iqn.2026-11.icn.estga:servidor/tpg1/luns create /backstores/fileio/disco0
/> iscsi/iqn.2026-11.icn.estga:servidor/tpg1/acls create iqn.2026-11.icn.estga:
    cliente
/> saveconfig
/> exit
```

Traduzindo: criámos um *backstore* (o ficheiro que serve de disco), um *target* identificado por um IQN, uma *LUN* (a unidade que o cliente verá) e uma *ACL* que autoriza um cliente específico, também identificado por IQN.

Atenção

Os IQNs do ACL (no servidor) e do initiator (no cliente) **têm de coincidir exatamente**. É aqui que a maioria dos erros acontece. Copie com cuidado.

3.3 Configurar o cliente

[CLI]

```
$ sudo nano /etc/iscsi/initiatorname.iscsi
```

Substitua a linha existente por:

```
InitiatorName=iqn.2026-11.icn.estga:cliente
```

[CLI]

```
$ sudo systemctl restart iscsid open-iscsi  
$ sudo iscsiadm -m discovery -t sendtargets -p <IP-do-servidor>
```

Saída esperada

```
192.168.1.50:3260,1 iqn.2026-11.icn.estga:servidor
```

```
$ sudo iscsiadm -m node --targetname iqn.2026-11.icn.estga:servidor --login
```

Saída esperada

```
Login to [iface: default, target: iqn.2026-11.icn.estga:servidor, portal: ...] successful.
```

3.4 Usar o disco

[CLI]

```
$ lsblk
```

Procure um disco novo de 500M (ex. sdb).

Atenção

Confirme bem qual é o disco novo antes do próximo comando. Formatar o disco errado destrói a VM.

[CLI]

```
$ sudo mkfs.ext4 /dev/sdX  
$ sudo mkdir -p /mnt/iscsi  
$ sudo mount /dev/sdX /mnt/iscsi  
$ echo "ola via iscsi" | sudo tee /mnt/iscsi/teste.txt  
$ df -h /mnt/iscsi
```

Confirme antes de avançar

Repare no que teve de fazer que **não** fez no NFS: **formatar**. Recebeu um dispositivo de bloco em bruto, e foi o *cliente* que lhe criou um sistema de ficheiros.

3.5 Porque não pode partilhar este LUN

Atenção

Não faça isto: montar este mesmo LUN em escrita a partir de um segundo cliente ao mesmo tempo.

No NFS funcionou porque o servidor coordena os acessos. No iSCSI não há coordenação nenhuma: cada cliente teria a sua própria cache e as suas próprias estruturas de ext4 em memória, sem saber da existência do outro. O resultado seria a corrupção do sistema de ficheiros.

Para permitir escrita simultânea sobre um dispositivo de bloco partilhado seria necessário um **sistema de ficheiros de cluster** (GFS2, OCFS2), com bloqueio distribuído.

	NFS (ficheiro)	iSCSI (bloco)
O que o cliente recebe	Uma pasta	Um disco em bruto
Quem formata	Já estava formatado	O cliente
Quem gere o FS	O servidor	O cliente
Vários clientes a escrever	Sim	Não (sem FS de cluster)

4 Parte D – Object storage com MinIO

[SRV]

```
$ sudo mkdir -p /srv/minio-data
$ docker run -d --name minio -p 9000:9000 -p 9001:9001 \
  -e "MINIO_ROOT_USER=admin" \
  -e "MINIO_ROOT_PASSWORD=minioadmin123" \
  -v /srv/minio-data:/data \
  minio/minio server /data --console-address ":9001"
$ docker ps | grep minio
```

Abra a consola web em <http://<IP-do-servidor>:9001>.

4.1 Cliente mc

[SRV]

```
$ curl -O https://dl.min.io/client/mc/release/linux-amd64/mc
$ chmod +x mc
$ ./mc alias set localminio http://localhost:9000 admin minioadmin123
```

4.2 Bucket e objetos

```
$ ./mc mb localminio/dados-aula11
$ echo "objeto de teste" > objeto.txt
$ ./mc cp objeto.txt localminio/dados-aula11/objeto.txt
$ ./mc ls localminio/dados-aula11
$ ./mc cp localminio/dados-aula11/objeto.txt objeto-descarregado.txt
$ cat objeto-descarregado.txt
```

Confirme antes de avançar

Não montou nada. Não formatou nada. O acesso foi por **API HTTP** – um modelo completamente diferente dos dois anteriores.

Confirme que o objeto é apenas um ficheiro no servidor:

```
$ sudo ls -R /srv/minio-data/
```

5 Parte E – NFS como Persistent Volume (opcional)

Requer o cluster da Aula 05.

Instale o cliente NFS em **todos os nós**:

```
$ sudo apt install -y nfs-common
```

Crie e aplique o `nfs-pv.yaml` do Guião 11:

```
$ kubectl apply -f nfs-pv.yaml
$ kubectl get pv,pvc
```

Saída esperada

```
NAME CAPACITY ACCESS MODES STATUS CLAIM
nfs-pv 1Gi RWX Bound default/nfs-pvc
```

RWX é **ReadWriteMany** – vários nós podem escrever ao mesmo tempo. Só é possível porque o NFS é armazenamento *ao nível de ficheiro*. Com iSCSI teria de usar **ReadWriteOnce**, pela razão que viu na Parte C.

Crie os dois Pods do guião e confirme:

```
$ kubectl exec leitor -- cat /dados/k8s.txt
```

E no servidor NFS:

```
$ cat /srv/nfs/partilha/k8s.txt
```

6 Limpeza

[CLI]

```
$ sudo umount /mnt/iscsi
$ sudo iscsiadm -m node --targetname iqn.2026-11.icn.estga:servidor --logout
$ sudo umount /mnt/nfs
```

[SRV]

```
$ sudo umount /mnt/nfs-local
$ docker rm -f minio
```

7 Problemas frequentes

mount.nfs: access denied by server while mounting

O IP do cliente não está abrangido pela sub-rede em `/etc/exports`. Confirme o IP real do cliente e a sub-rede que escreveu. Depois de corrigir: `sudo exportfs -ra`.

mount.nfs: Connection timed out

O servidor NFS não está a correr, ou há firewall pelo meio. Teste:

```
$ sudo systemctl status nfs-kernel-server
$ showmount -e <IP-do-servidor>
```

Escrevo no NFS mas dá Permission denied

Problema de permissões ou de mapeamento de UIDs. Para efeitos de laboratório, o `chmod 777` da secção 2.1 resolve. Em produção usar-se-ia consistência de UIDs ou Kerberos.

iscsiadm: No portals found

O target não está a correr ou o IP está errado. No servidor:

```
$ sudo systemctl status rtslib-fb-targetctl
$ sudo targetcli ls
```

Erro 24 no iscsiadm: *iSCSI login failed due to authorization failure*

Os IQNs não coincidem. Compare exatamente:

```
# no cliente:
$ cat /etc/iscsi/initiatorname.iscsi
# no servidor:
$ sudo targetcli ls /iscsi/iqn.2026-11.icn.estga:servidor/tpg1/acls
```

O disco iSCSI não aparece no lsblk depois do login

Aguarde alguns segundos e repita. Se persistir, force a deteção:

```
$ sudo iscsiadm -m session --rescan
```

A configuração do targetcli desaparece após reiniciar

Faltou o `saveconfig`. Volte a entrar no `targetcli` e execute-o.

O MinIO não arranca

Veja os logs:

```
$ docker logs minio
```

A password tem de ter pelo menos 8 caracteres – é um requisito do MinIO.

```
./mc: Permission denied
```

Faltou o `chmod +x mc`.

O PVC fica em Pending

O PV e o PVC têm de ter modos de acesso e capacidade compatíveis, e o PVC precisa de `storageClassName:` para ligar a um PV estático. Investigue:

```
$ kubectl describe pvc nfs-pvc
```

8 Referência rápida

Comando	Para que serve
<code>sudo exportfs -ra</code>	Reaplicar as exportações NFS
<code>sudo exportfs -v</code>	Ver o que está exportado
<code>showmount -e <ip></code>	Ver exportações de um servidor
<code>mount <ip>:/caminho /mnt/x</code>	Montar partilha NFS
<code>sudo targetcli</code>	Configurar targets iSCSI
<code>targetcli ls</code>	Ver a configuração atual
<code>iscsiadm -m discovery -t sendtargets -p <ip></code>	Descobrir targets
<code>iscsiadm -m node --targetname X --login</code>	Ligar a um target
<code>iscsiadm -m session</code>	Ver sessões ativas
<code>mc alias set NOME URL user pass</code>	Configurar acesso ao MinIO
<code>mc mb alias/bucket</code>	Criar bucket
<code>mc cp origem destino</code>	Copiar objetos
<code>mc ls alias/bucket</code>	Listar conteúdo

9 Checklist final

- ☐ Identifiquei os tipos de storage de rede suportados pelo Proxmox
- ☐ Sei explicar a relação entre storage partilhado e migração ao vivo
- ☐ Configurei o NFS e montei a partilha no cliente
- ☐ **Testei o acesso concorrente ao NFS com sucesso**
- ☐ Configurei o target iSCSI e liguei-me a partir do initiator
- ☐ Formatei e montei o disco iSCSI, e percebo porque foi preciso formatar
- ☐ Sei explicar porque não posso partilhar o LUN iSCSI entre dois clientes
- ☐ Corri o MinIO, criei um bucket e enviei/obtive um objeto

- ☐ (Opcional) Criei o PV NFS no Kubernetes e percebo o **ReadWriteMany**
- ☐ Desmontei tudo e removi o container do MinIO